AI-Based Natural Language to SQL Query System

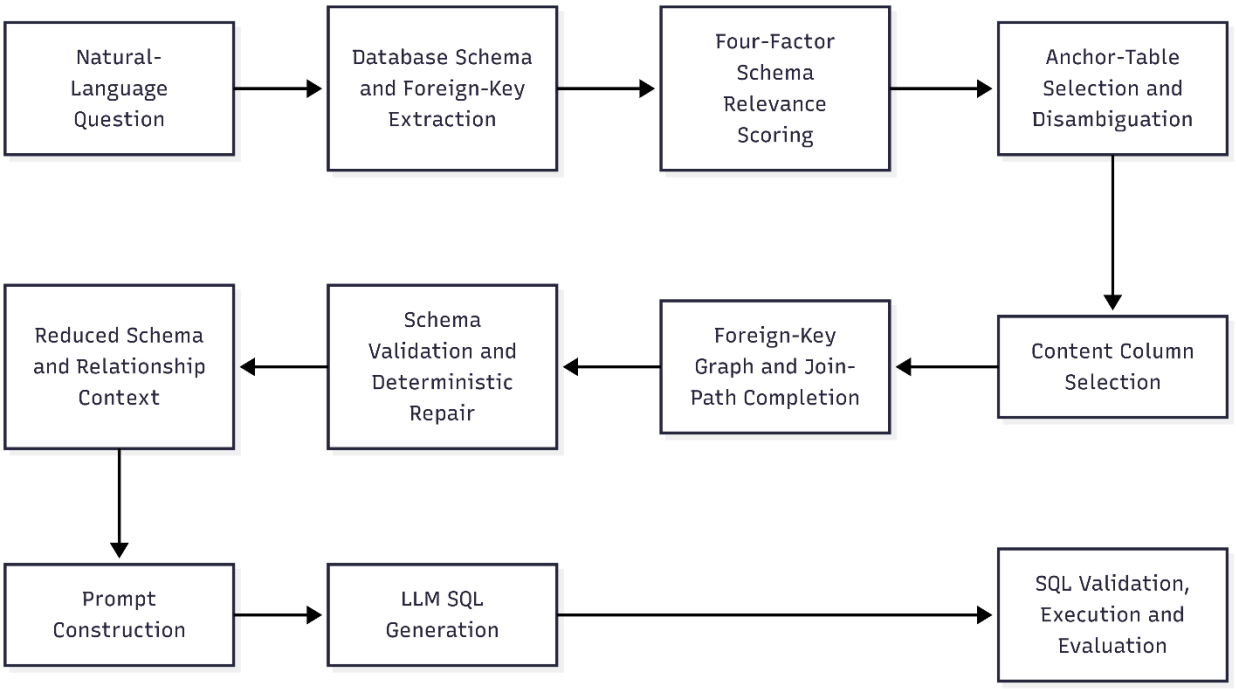
Latency and Accuracy Aware Schema Linking for LLM-Based Text-to-SQL Generation

1. Introduction

Large Language Models (LLMs) can convert natural-language questions into SQL queries. However, providing the complete database schema for every question can increase prompt size and include irrelevant tables and columns. This may increase processing overhead and make it harder for the model to identify relevant schema elements.

This project focuses on **schema linking and schema reduction**: selecting the tables, columns, and relationships relevant to a question before passing them to an LLM. The method combines lexical similarity, semantic similarity, data-type compatibility, and foreign-key relationships. SQL generation is the downstream task; the main contribution is optimizing the schema context supplied to the model.

2. System Architecture



The database module extracts tables, columns, data types, and foreign-key relationships. The schema linker scores candidate columns, selects relevant tables and columns, and preserves foreign-key paths needed to connect them. Validation checks the reduced

schema before it is added to the prompt. The LLM then generates SQL using this smaller context.

3. Formula Used

The base relevance score is:

$$S_{\text{base}} = w_{\text{lex}} \times S_{\text{lex}} + w_{\text{sem}} \times S_{\text{sem}} + w_{\text{type}} \times S_{\text{type}}$$

Using the configured weights:

$$S_{\text{base}} = 0.40 \times S_{\text{lex}} + 0.35 \times S_{\text{sem}} + 0.15 \times S_{\text{type}}$$

Relationship and graph-proximity evidence is applied additionally during table selection, disambiguation, and join-path completion.

Explanation of Terms

- **S_lex — Lexical similarity:** Measures word overlap between the question and table/column names. Identifier splitting and stemming help match related word forms.
- **S_sem — Semantic similarity:** Measures similarity in meaning between the question and schema-column descriptions using all-MiniLM-L6-v2 embeddings.
- **S_type — Data-type compatibility:** Checks whether a column's data type suits the question, such as a numeric column for a minimum or maximum value.
- **w_lex, w_sem, w_type — Weights:** Control each score's contribution. The configured values are 0.40, 0.35, and 0.15.
- **Relationship/graph evidence:** Uses foreign-key connections to distinguish similarly named columns and retain intermediate tables needed to connect selected tables. This is additional selection logic, not a separate term in the base formula.

The system uses a content-column budget of eight. Foreign-key columns required for joins are preserved separately, and validation repairs missing structural links where possible.

4. Technology Stack

The system follows a pipeline in which schema optimization occurs before LLM inference.

Technology	Purpose
Python	Core algorithm and application logic
SQLite	Database metadata extraction and query evaluation
Google Gemini (`gemini-3.5-flash-lite`)	SQL Query generation
SentenceTransformers (`all-MiniLM-L6-v2`)	Semantic embeddings for schema columns
NumPy and scikit-learn	Numerical and similarity calculations
Pytest	Automated testing
Vanilla JavaScript and Python HTTP server	Local web interface

5. Dataset and Results

The system was evaluated on **25 questions across five databases** from the Spider Text-to-SQL benchmark.

Database	Domain	Tables	Total Columns	Total Rows
hr_1	Human resources	7	35	216
car_1	Automotive specifications	6	23	890
flight_2	Airlines and flights	3	13	1,312
world_1	Global demographics	3	24	5,302
sakila_1	Media rental store	16	89	46,823

Overall Results

Metric	Full-Schema Baseline	Proposed Pipeline
Prompt Tokens	757.08	361.92
Columns Sent	36.80	7.84
Schema Reduction	0.0%	73.40%
Algorithm Linking Latency	0.0 ms	36.2 ms (avg)
LLM Generation Latency	3.00 s	2.90 s
End-to-End (Total) Latency	3.00 s	2.93 s
Schema Linking Precision	9.19%	36.02%
Schema Recall Accuracy	100.0%	92.67%
Schema F1-Score	16.37%	46.74%
Generation Throughput	6.84 tps	7.10 tps

Meaning of the Metrics

- **Prompt tokens:** Units of text processed by the LLM. Fewer tokens mean a smaller prompt.
- **Columns sent:** Number of columns sent to the LLM.
- **Schema reduction:** Percentage decrease in the number of schema columns supplied compared with the full-schema baseline.
- **Algorithm Linking Latency:** Time taken by the schema linker to select the reduced schema. The 36.2 ms figure is the warm average, excluding first-time embedding-model initialization.
- **LLM Generation Latency:** Time taken by the LLM to generate the query.
- **End-to-end Latency:** Time for the overall process, including algorithm and LLM latency.
- **Schema Linking Precision:** Of the schema elements selected, the proportion that were actually required.
- **Schema Recall Accuracy:** Of all schema elements required by the reference query, the proportion retained by the linker.
- **F1-score:** The harmonic mean of precision and recall. it balances selecting relevant elements with retaining required ones.
- **Generation Throughput:** Number of tokens per second (tps) generated by the LLM as output.

Key Findings

- Average prompt tokens decreased by **52.2%**, from 757.08 to 361.92.
- The average number of columns supplied decreased by **73.4%**, from 36.80 to 7.84.
- Macro precision increased from **9.19% to 36.02%**, and macro F1-score increased from **16.37% to 46.74%**.
- Macro recall was **92.67%**, meaning most required schema elements were retained, although some were missed.
- Warm schema-linking latency averaged **36.2 ms**. The first query had a separate cold-start cost of approximately **26.06 seconds** while the embedding model loaded.

The evaluation is limited to 25 questions and five databases, so the results are an initial benchmark rather than proof of general performance. Actual API billing savings were not measured; token reduction is the measured efficiency indicator.

6. Conclusion

The project implements a schema-linking optimization layer for LLM-based Text-to-SQL systems. By combining lexical and semantic similarity, data-type compatibility, and foreign-key graph information, it supplies a smaller and more relevant schema to the LLM while preserving most required schema elements. Across 25 questions from five Spider databases, the system reduced prompt tokens by 52.2% and schema columns by 73.4%, achieved 92.67% macro recall, and improved macro F1-score from 16.37% to 46.74%. Further evaluation on larger datasets is needed to assess generalization.

References

1. Yu, T. et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task.” EMNLP, 2018. https://aclanthology.org/D18-1425/
2. “Re-examining the Role of Schema Linking in Text-to-SQL.” EMNLP, 2020. https://aclanthology.org/2020.emnlp-main.564/
3. Wang, B. et al. “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers.” ACL, 2020. https://aclanthology.org/2020.acl-main.677/
4. “RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL.” AAAI, 2023. https://ojs.aaai.org/index.php/AAAI/article/view/26535/
5. Li, J. et al. “Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs.” 2023. https://arxiv.org/abs/2305.03111